Operating Systems Lab 7

Processes, redirection and pipes

Study brief for comprehension and oral defense. This supplements the formal UTM report and does not replace its required answers or evidence.

What the lab asks

Follow an ordinary command through process creation, program replacement and waiting. Then recreate two shell behaviors directly: redirect standard output into a file, and connect one command's output to another command's input. The four supplied programs share one idea: a process acts through kernel-managed identities and connections.

Plain terms

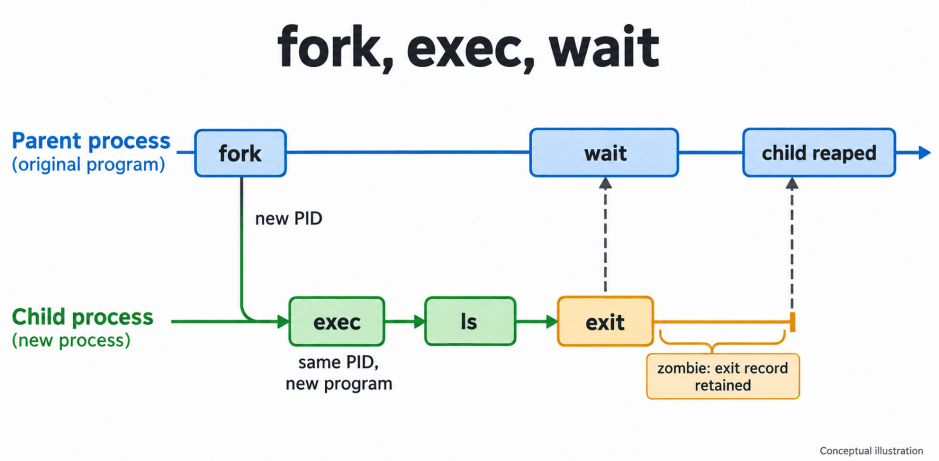
Program: executable instructions on disk. **Process:** one running instance with a PID, state, memory and descriptors. **fork:** create a child process. **exec:** replace a process's program image. **waitpid:** collect a child's termination status. **Descriptor:** a small per-process number naming an open file-like connection.

Read in ten minutes

2 min: sketch fork -> exec -> exit -> wait. 2 min: trace run.c:12-25. 2 min: explain exit versus zombie collection. 2 min: follow descriptor 1 into out.txt. 2 min: draw the pipe ends, predict the leaked-writer result, and compare with the bounded observation.

Process lifetime

The child keeps its PID across exec. Exit ends execution; waitpid later collects the remaining status.



Trace one execution

Worked source trace: run.c

The shell starts the parent. `fork()` returns 0 in the child and the child's PID in the parent (lines 12-15). The child prints its identity, then `execlp("ls", "ls", "-l", ...)` searches PATH and replaces its program image (lines 16-18). Success never returns to the old code. If lookup fails, `perror` runs and the child explicitly exits 127 (lines 19-20). The parent blocks in `waitpid` and interprets the status (lines 23-25).

What the captured run showed

On 19 Oct 2026, the recorded missing-command variant printed `No such file or directory` and the parent reported status 127. Parent 1795 and child 1796 are values from that run, not expected constants. The report's Part 1 is pages 5-6; the complete transcript is `../lab78/execution-session-2026-10-19.txt`.

Zombie distinction

In `zombie.c`, the child runs `ls` and exits while the parent sleeps. During that interval ps showed child state Z. The later waitpid collected it and the next ps found no matching rows. A zombie has finished executing; the kernel retains its PID and status until collection. See report Part 2, pages 8-9.

Source route

`run.c:12-25`; `zombie.c:12-26`; report Part 1 pages 5-6 and Part 2 pages 8-9.

Report and source anchors

Part 1: process creation and exec, report pages 5-6. **Part 2:** zombie and waitpid, pages 8-9. **Sources:** run.c and zombie.c. The evidence transcript records the commands and observed values.

Follow the connections

Worked connection: redirection and a pipe

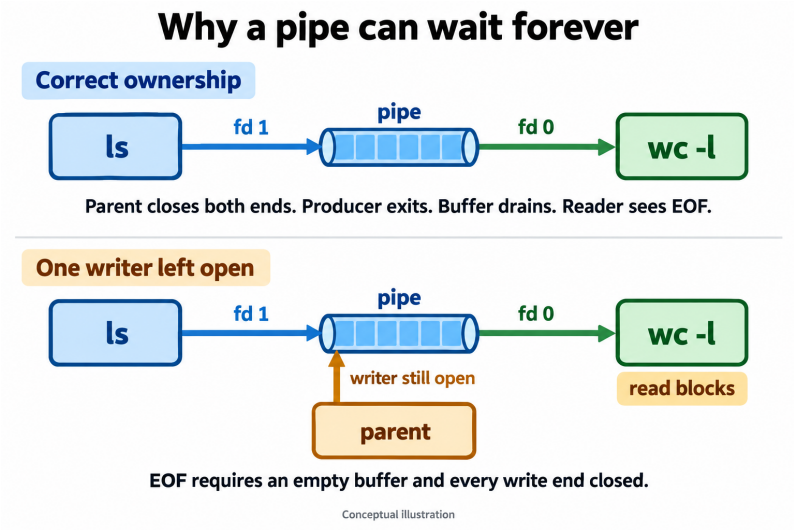
`redirect.c` closes descriptor 1, then opens `out.txt`. Since 1 is the lowest unused number, open returns 1. The child writes its diagnostic to descriptor 2, then execs ls; ls writes through descriptor 1 into the file. The parent's descriptor table is unchanged. See report Part 3, pages 10-11. `pipeline.c` creates a pipe: p[0] is its read end, p[1] its write end. The ls child connects stdout to p[1], and wc connects stdin to p[0]. The parent and children close every unused copy. EOF arrives only after buffered bytes drain and all writers close. The captured `ls /etc | wc -l` result was 172 for both manual and shell pipelines; the three-stage example gave 108 for both. Those are environment-specific counts from one session.

Changed condition

The deliberately broken copy retained the parent's p[1]. After ls exited, wc remained alive because a writer reference still existed; the isolated process group was terminated. Never run this unbounded. See report Part 4, pages 12-14 and the execution record.

Pipe ownership

The diagram is conceptual. Its retained writer is enough to prevent EOF even after the intended command exits.



This is a concept illustration, not a captured process screen. The measured example and bounded cleanup are in the formal report and shared execution record.

Oral practice prompts

Answer these aloud before turning to the answer pages. Questions 1-15 cover the main required mechanisms; 16-20 test changed conditions and the limits of the evidence.

1. [Likely] After fork, what does the child receive, what does the parent receive, and which branch runs?

2. [Likely] Why does run.c flush stdout immediately before fork?

3. [Likely] Explain program versus process using the child in run.c.

4. [Likely] What does execlp search, and what does its first argument after the file name become?

5. [Likely] Why is the perror line reached only when the requested program cannot be executed?

6. [Likely] How should a parent establish that a child exited normally before reading its exit code?

7. [Likely] Why does the missing-command version report status 127 in this lab?

8. [Likely] What relationship do getpid and getppid show in the captured run, and are those numbers fixed?

9. [Likely] A ps row shows state Z for the child in zombie.c. Is ls still executing?

10. [Likely] Why does zombie.c sleep before waitpid, and does that guarantee every observer sees a zombie?

11. [Likely] What action removes the zombie row in the captured observation?

12. [Likely] What do descriptors 0, 1 and 2 conventionally mean?

13. [Likely] Why does open return descriptor 1 after redirect.c closes descriptor 1?

14. [Likely] Why does the open diagnostic stay on screen while the ls listing goes into out.txt?

15. [Likely] What are O_CREAT, O_TRUNC and 0644 doing in redirect.c?

16. [Harder] If a child exits because a signal killed it, can the parent interpret the status as an ordinary exit code?

17. [Harder] Does status 127 prove that the executable name was absent from PATH?

18. [Harder] The parent closes its pipe writer, but wc still waits. What single leak could explain it?

19. [Harder] In the measured broken run, why is wc waiting while ls has already finished?

20. [Harder] Both pipeline counts were 172. What does that demonstrate, and what does it not prove?

Checking answers: questions 1-10

1. The child receives 0 and takes the child branch. The parent receives the child's positive PID and continues to wait.

Why: The return value is the same event viewed in two process contexts; it is how one source line selects two paths.
Change: If fork fails, it returns a negative value and neither normal child nor parent-success path should be treated as valid.
Trace: run.c:12-15; report Part 1, pp. 5-6.

2. It prevents text still buffered in the parent's stdio stream from being copied into the child and printed twice later.

Why: fork duplicates memory state as well as creating a process, so an unflushed user-space buffer is copied.
Change: If output is already unbuffered or the buffer is empty, this particular duplicate-output risk is absent.
Trace: run.c:9-12; report Part 1, p. 5.

3. A program is executable instructions such as ls. A process is a running instance with a PID, memory, state and descriptors. The child process starts as a copy and later runs ls.

Why: The PID belongs to the process identity, not to a filename.
Change: After successful exec the program changes, but the same child process and PID continue.
Trace: run.c:12-20; report Part 1, pp. 5-6.

4. execlp searches PATH when the file name has no slash. The next argument becomes argv[0], conventionally the program name passed to the new image.

Why: The executable lookup name and argument vector are separate inputs to exec.
Change: An absolute path such as /bin/ls avoids PATH search; argv[0] can still be a chosen string.
Trace: run.c:18; report Part 1, p. 5.

5. Successful exec replaces the child image and does not return. A failed exec returns -1, so the next source line reports the error.

Why: The return path itself distinguishes success from failure.
Change: If the path exists but permission or format prevents execution, exec can still fail and return.
Trace: run.c:18-20; report Part 1, pp. 5-6.

6. It checks WIFEXITED(status) first, then uses WEXITSTATUS(status). The encoded wait status can also describe signal termination.

Why: The integer returned through waitpid contains status information, not always a plain exit value.
Change: If WIFSIGNALED(status) is true, inspect WTERMSIG instead of treating WEXITSTATUS as a normal result.
Trace: run.c:23-25; report Part 1, p. 6.

7. The source explicitly calls exit(127) after execlp fails. The number is the program's chosen error status, not a value returned by exec.

Why: exec reports failure by returning; the child code decides how to terminate afterward.
Change: Changing the explicit exit value changes what the parent observes. xv6 Lab 8 uses 1 for its own failure path.
Trace: run.c:19-20; report Part 1, pp. 5-6; measured run, p. 6.

8. They identify the child and its current parent. The captured run used parent 1795 and child 1796; those PIDs are one run's values, not constants.

Why: The calls report runtime process identities assigned by the operating system.
Change: A later run will normally receive different PIDs; reason from parent-child relationships instead of memorizing numbers.
Trace: run.c:9-16; report Part 1, p. 6; execution session, 19 Oct 2026.

9. No. The child has exited. The kernel retains its PID and exit information until the parent collects it with waitpid.

Why: A zombie is a bookkeeping state after execution, not a runnable process consuming CPU.
Change: If the parent had already waited, the child row would have been reaped and no longer appear.
Trace: zombie.c:12-25; report Part 2, pp. 8-9; execution session, 19 Oct 2026.

10. The delay widens the interval in which the child has exited while the parent has not waited. It improves the chance of observation but cannot make scheduling or a separate ps command perfectly deterministic.

Why: The state exists only between child exit and parent collection.
Change: Without the delay the same state may be too brief to catch; after waitpid it is gone.
Trace: zombie.c:22-25; report Part 2, pp. 8-9.

Checking answers: questions 11-20

11. The parent calls waitpid, receives the child's termination status, and lets the kernel release the remaining process record. The later ps found no matching rows.

Why: Process termination and parent collection are separate events.
Change: If the parent never waits, an init-like process may eventually adopt and reap the child after the parent exits; timing depends on the environment.
Trace: zombie.c:24-26; report Part 2, pp. 8-9; execution session, 19 Oct 2026.

12. They are a process's standard input, standard output and standard error. Each number refers to an open file-like connection, not a specific device.

Why: The same program can read or write without knowing whether a descriptor points to a terminal, file or pipe.
Change: A shell or a program can redirect any of these connections before the target program runs.
Trace: redirect.c:11-16; report Part 3, pp. 10-11.

13. open allocates the lowest unused descriptor in that process. Since 1 was just freed, it is allocated to out.txt.

Why: Descriptor numbers are table indices, and open chooses an available one.
Change: If descriptor 1 remained open, open would normally choose another free number and ls would still write to the old stdout.
Trace: redirect.c:11-14; report Part 3, pp. 10-11.

14. The diagnostic uses stderr, descriptor 2, which remains attached to the terminal. The child replaces descriptor 1 with the file before execing ls.

Why: stdout and stderr are separate connections even when both originally lead to the same terminal.
Change: Closing and replacing descriptor 2 instead would redirect diagnostics while leaving normal output on descriptor 1.
Trace: redirect.c:12-16; report Part 3, pp. 10-11; execution session, 19 Oct 2026.

15. O_CREAT creates out.txt if absent; O_TRUNC empties an existing file; 0644 requests owner read/write and group/other read permissions, modified by the process umask.

Why: The open flags control file creation and replacement, while the mode supplies requested permission bits.
Change: Without O_TRUNC, an older longer file could retain trailing bytes after a shorter new listing.
Trace: redirect.c:13; report Part 3, p. 10.

16. No. It should first test WIFEXITED. For signal termination, test WIFSIGNALED and inspect WTERMSIG.

Why: waitpid reports more than one termination mode in its encoded status.
Change: A normal exit permits WEXITSTATUS; a signal death needs the signal-specific branch.
Trace: run.c:23-25; report Part 1, p. 6.

17. It proves only that this source's failed-exec path chose exit(127). execlp can fail for other reasons, such as permission or format errors.

Why: The status value is assigned after the generic exec failure; it does not encode the exact errno.
Change: Check the perror text or errno to distinguish a missing file from another execution failure.
Trace: run.c:18-20; report Part 1, p. 6.

18. Another process, including the ls child, still holds a copy of the write end. The reader cannot see EOF until all write references are closed and buffered bytes are consumed.

Why: EOF depends on ownership of every writer reference, not only the parent's descriptor table.
Change: Close the unused duplicate in every process; closing only the parent copy is insufficient.
Trace: pipeline.c:12-35; report Part 4, pp. 12-14.

19. The parent intentionally retained p[1], so wc received available bytes but never EOF. The parent waited for children, while wc remained alive; the isolated process group was then terminated.

Why: A pipe reader blocks after buffered data is exhausted whenever any writer remains open.
Change: Close every parent and child copy of p[1], and wc can finish after ls exits and the buffer drains.
Trace: pipeline.c:32-35; report Part 4, pp. 12-14; execution session, 19 Oct 2026.

20. The manual pipeline and shell pipeline produced the same line count for /etc in that captured environment. It does not prove a permanent count, unchanged input between runs, or correctness for all data.

Why: The test compares two mechanisms under one directory state and one execution session.
Change: If /etc changes between runs, equal results are no longer guaranteed even when both implementations are correct.
Trace: pipeline.c:9-35; report Part 4, pp. 12-14; execution session, 19 Oct 2026.